

Manuale d'uso — Claude Design ↔ Earth Defense

Guida completa per disegnare animazioni SVG su **claude.ai/design** e farle implementare nel gioco **Earth Defense**. Pensata per essere usata da sola, senza conoscenze tecniche pregresse.

Versione del documento: 1.0 — progetto Earth Defense.

1. Cos'è e a cosa serve

Earth Defense usa molti elementi grafici in formato **SVG** (immagini vettoriali): velivoli, l'UFO, barre della vita, icone, badge, effetti. Finora questi asset venivano realizzati e inseriti a mano nel codice.

Questo "ponte" collega due strumenti:

- **claude.ai/design** — un'app web (un canvas di design) dove tu, insieme a Claude lì dentro, **disegni e vedi in anteprima** le animazioni SVG.
- **Earth Defense** (qui, in Claude Code) — dove io **prelevo** quelle animazioni e le **cablo nel gioco** seguendo le sue regole tecniche.

Il ponte funziona nei **due sensi**:

1. **Io → Design**: carico nel progetto di design le *regole del gioco*, così quando disegni con Claude le animazioni nascono già compatibili.
2. **Design → me**: per ogni animazione, Claude Design produce anche un piccolo file di istruzioni (il *manifest*) che mi dice esattamente come collegarla al gioco.

Importante: **non è una sincronizzazione automatica**. Tu disegni con calma sul canvas; quando sei pronto, mi dici di portare dentro l'animazione e io la prelevo *su richiesta*.

2. Setup iniziale (si fa una volta sola)

Questi passi creano il collegamento. Li eseguo io; a te tocca solo autorizzare e, se richiesto, fare login.

1. **Autorizzazione design**. La prima volta che accedo ai progetti di design, ti comparirà una richiesta di permesso per collegare l'accesso "design-system" al tuo account claude.ai. Va concessa una volta sola. Se la sessione non ha un login attivo, userò la procedura di login dedicata.
2. **Creazione del progetto**. Creo un progetto chiamato, ad esempio, **"Earth Defense — Animazioni"**. Da quel momento puoi aprirlo nel browser su claude.ai/design.
3. **Caricamento delle regole**. Carico nel progetto il documento delle regole di disegno (`EARTH-DEFENSE-ASSET-RULES.md`). È ciò che permette a Claude, sul canvas, di sapere come vanno fatti gli asset del gioco.

4. **Esempi di riferimento.** Carico anche alcuni asset reali del gioco come esempi da imitare o remixare: l'UFO e una barra della vita (esempi "ricchi"), e il caccia F-22 (esempio "3D").

Finito questo, il setup non va più rifatto. L'unica manutenzione è ricaricare le regole se in futuro cambiano (vedi sezione 8).

3. Disegnare su claude.ai/design

Quando disegni un'animazione, esiste **una distinzione fondamentale** che decide cosa puoi usare. Ogni asset appartiene a una delle due "rotte".

Rotta "DOM" — animazioni ricche (la più comune)

Per interfaccia, barre, badge, effetti, l'UFO: tutto ciò che vive "sopra" la scena.

Puoi usare liberamente:

- gradienti e bagliori;
- filtri (sfocature, effetti);
- animazioni fluide via CSS (movimenti in loop, pulsazioni, particelle).

Devi però rispettare:

- dare un **nome stabile** (id) a ogni parte che il gioco deve animare o accendere;
- prevedere uno **stato di riposo** statico e uno **stato attivo** acceso da una classe (per convenzione `on`): es. raggio spento a riposo, acceso quando il gioco lo richiede;
- **sfondo trasparente**;
- **niente testo** scritto dentro l'immagine (nomi, etichette, numeri): li mette il gioco.

Rotta "Mesh" — unità 3D sul globo

Per velivoli e oggetti che volano sul mappamondo (come il caccia). Qui il gioco trasforma il disegno in un modello 3D, e accetta **solo forme a tinta piatta**.

Regole rigide:

- **solo colori pieni**: niente gradienti, filtri, animazioni dentro l'SVG;
- **muso del velivolo rivolto verso l'alto** nel disegno;
- ogni parte che dovrà animarsi (luci, fiamme dei motori) su una **forma separata con un nome**. Le animazioni le aggiunge il gioco nel suo motore di render.

Regola pratica

Se non sei sicuro, disegna la versione **ricca** (rotta DOM): è sempre recuperabile. Ma se stai disegnando chiaramente un velivolo 3D, segui da subito la rotta Mesh.

4. I due "contratti" che accompagnano ogni asset

Perché io possa cablare un'animazione senza tirare a indovinare, ogni SVG arriva con due informazioni standard. Claude Design le produce automaticamente seguendo le regole caricate.

Contratto 1 — l'intestazione

La prima riga dell'SVG è un commento che riassume l'asset. Esempio:

```
<!-- ed-asset route="dom" name="nuovo-effetto" animatable="#beam,#glow" state-class="on"
i18n-text="none" -->
```

Mi dice in un colpo d'occhio: che rotta è, come si chiama, quali parti si animano, quale classe accende lo stato attivo, e se c'è testo (di norma nessuno).

Contratto 2 — il manifest

Accanto a `nuovo-effetto.svg` c'è un file `nuovo-effetto.integration.json`: la guida completa al cablaggio. In parole semplici mi dice, per ogni parte animata:

- **cosa fa** (es. "raggio traente", "alone");
- **chi la muove**: se si anima da sola via CSS (`asset`) o se deve pilotarla il gioco (`game`);
- **come**: accendere una classe, scalare, cambiare colore via variabile, ecc.;
- quali **manopole** deve esporre il gioco (scala, colore, riempimento);
- **dove** conviene inserirla nel gioco (un overlay sul globo, un pannello, un'unità 3D).

Non devi scrivere tu questi file: li genera Claude Design. Ti basta sapere che esistono e che rendono l'integrazione rapida e affidabile.

5. Il workflow di tutti i giorni

Il ciclo completo, una volta fatto il setup:

1. **Disegni** l'animazione su `claude.ai/design`, iterando con Claude finché ti piace, e la salvi nel progetto (es. nella cartella `animations/`).
2. **Me lo dici** qui, con una frase semplice: «porta dentro l'animazione `nome`».
3. **Io la prelevo**: leggo l'SVG e il suo manifest dal progetto di design.
4. **La cablo nel gioco** seguendo le istruzioni del manifest e i pattern già esistenti, e mi occupo delle traduzioni dei testi eventuali (italiano e inglese).
5. **Verifico** che funzioni avviando il gioco e lanciando i test.
6. Quando è tutto a posto, l'animazione è nel gioco.

Tu intervieni solo nei passi 1 e 2. Il resto è a carico mio.

6. Frasi da dirmi (il "telecomando")

Non servono comandi tecnici: bastano frasi in linguaggio naturale. Le più utili:

- **«Porta dentro l'animazione `nome` .»** — Prelevo quell'asset dal progetto design e lo integro nel gioco.
 - **«Aggiorna le regole sul progetto design.»** — Se abbiamo cambiato le convenzioni del gioco, rigenero e ricarico il documento delle regole sul canvas.
 - **«Riallinea gli esempi.»** — Ricarico sul progetto design gli asset attuali del gioco come riferimenti aggiornati.
 - **«Elenca le animazioni sul progetto design.»** — Ti dico cosa c'è attualmente nel progetto.
 - **«Quest'animazione non va: dimmi cosa correggere.»** — Controllo un asset e ti spiego cosa sistemare sul canvas perché diventi compatibile.
-

7. Risoluzione problemi

- **La richiesta di autorizzazione non compare / accesso negato.** Serve il collegamento dell'account claude.ai con l'accesso design. Riprovo l'operazione; se necessario, avvio la procedura di login dedicata e ti guido.
 - **"Progetto di tipo sbagliato".** Il ponte funziona solo su progetti di tipo *design-system*. Se ne è stato creato uno normale, ne creo uno nuovo del tipo giusto (il tipo non si può cambiare dopo).
 - **Un asset non si integra come previsto.** Quasi sempre è una regola non rispettata: testo dentro l'immagine, mancanza di id stabili, oppure una rotta "ricca" usata per un'unità 3D. Te lo segnalo con precisione e tu lo correggi sul canvas; poi lo riprelevo.
 - **L'animazione "ricca" è bellissima ma serve come unità 3D.** Le animazioni CSS e i gradienti non sopravvivono alla conversione 3D. O la rifai a tinte piatte (rotta Mesh), o la teniamo come overlay (rotta DOM). Te lo dico io quale strada conviene.
-

8. Manutenzione

- **Le regole del gioco cambiano?** Dimmi «aggiorna le regole sul progetto design»: rigenero il documento delle regole dalla fonte di verità nel codice e lo ricarico sul canvas, così Claude Design resta allineato.
 - **Sono cambiati gli asset di riferimento?** Dimmi «riallinea gli esempi» e ricarico le versioni aggiornate.
 - **Il documento delle regole** è versionato anche nel codice del gioco (`docs/EARTH-DEFENSE-ASSET-RULES.md`): è lì la fonte ufficiale; la copia sul canvas è uno specchio.
-

9. Glossario

- **SVG** — formato di immagine vettoriale (forme descritte da coordinate, non da pixel).
- **claude.ai/design** — l'app web dove disegni gli asset insieme a Claude.
- **Progetto design-system** — il "contenitore" sul canvas che ospita i tuoi asset e le regole.
- **DesignSync** — lo strumento con cui io leggo/scrivo quel progetto dal lato gioco.
- **Rotta DOM / overlay** — asset ricchi (gradienti, filtri, animazioni CSS) mostrati "sopra" la scena del gioco.
- **Rotta Mesh / 3D** — asset trasformati in modelli tridimensionali sul mappamondo; solo tinte piatte.
- **Intestazione (header)** — la riga di riepilogo all'inizio dell'SVG.
- **Manifest** — il file di istruzioni che accompagna ogni asset e spiega come collegarlo.
- **id / classe stabile** — un nome dato a una parte del disegno, così il gioco sa quale animare.
- **Stato** `.on` — la classe che accende lo stato attivo di un'animazione (es. il raggio).
- **i18n** — il sistema di traduzioni del gioco (italiano/inglese); per questo i testi non vanno scritti dentro gli asset.